SchoolNet — Flutter + FastAPI source handoff

This repository is an initial, runnable product foundation based on the supplied **SchoolNet Comprehensive PRD & Master Architecture Blueprint** and the execution instructions in `docs/SchoolNet_Build_Prompt.docx`. The supplied SchoolNet logo is included at `assets/schoolnet_logo.jpg` and referenced by the Flutter app.

Project structure

- `frontend/` — Android-first Flutter client, role selection, login/community signup, role-aware portal shell, API client, secure token storage, and offline feed caching.
- `backend/app/main.py` — FastAPI REST API, tenant-aware SQLAlchemy models, JWT/password handling, role-gated workflows, grade/section feeds, mark entry and rating calculation, structured messages, and Excel export.
- `backend/app/extensions.py` — subject/assignment setup, teacher appraisals, personal inbox, scoped marklist access, permitted staff messaging and DOCX report-card export.
- `backend/migrations/` — initial PostgreSQL schema and tenant-level RLS policies for Neon. Review before applying to a live database.
- `assets/schoolnet_logo.jpg` — the exact logo image supplied by the user.
- `docs/SchoolNet_Build_Prompt.docx` — Word-format build prompt derived from the pasted execution instructions and the PDF master plan.

Run the API locally

Bash

```bash
cd backend
python -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
export JWT_SECRET="replace-with-a-long-random-secret"
export DATABASE_URL="sqlite:///./schoolnet_dev.db"  # local demo only
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Interactive API reference: `http://localhost:8000/docs`; health check: `GET /health`.

Run the backend integration smoke test with `pip install -r requirements-dev.txt` followed by `pytest -q` from `backend/`. It exercises first-login password changes, teacher/student roles,

the 9.2-star formula, feed scoping, student privacy boundaries, DOCX report export and community inquiry routing.

For Neon, configure `DATABASE_URL` with a PostgreSQL connection string, set a strong `JWT_SECRET`, apply `migrations/001_initial.sql` followed by `migrations/002_rls.sql`, and use a dedicated non-owner runtime role with explicit grants. Do not use a database owner or superuser as the API runtime identity because those roles can bypass PostgreSQL RLS. The API sets `app.current_institution` transaction-locally from the authenticated JWT before tenant-scoped queries. The `institutions` directory is intentionally not RLS-protected so login can resolve a school slug; expose only the institution fields needed for public discovery in a production API.

The first institution and initial administrator must be provisioned by a trusted process. For a local SQLite demo, set the `SCHOOLNET_*` variables from `.env.example` and run `python -m app.seed_admin` from `backend/`. For Neon, provision through a trusted migration/operator identity. Do not expose an unauthenticated admin-creation endpoint. Institutional users created through the admin roster endpoint receive a one-time temporary PIN and are required to change it before accessing protected routes. Community users self-register against a school slug.

Run the Flutter app

Install Flutter and Android tooling, then from `frontend/`:

Bash

```
flutter pub get
flutter run
```

Set the backend origin at build time, for example:

Bash

```
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:8000
```

Use the machine/LAN address for a physical device; Android emulators commonly reach the host through `10.0.2.2`.

For a release APK, configure a production HTTPS API origin and signing credentials before building:

Bash

```
flutter build apk --release --dart-define=API_BASE_URL=https://api.example-scho
```

The development manifest alone permits local cleartext HTTP. Configure real release signing keys before distributing an APK; do not ship the scaffold's debug key.

Implemented in this foundation

- Five role identities: Administrator, Teacher, Student, Guidance & Counseling, and People/Community.
- Institutional email/password login; public community self-registration; first-login password-change enforcement; admin-created institutional users and CSV/XLSX roster import.
- School/class/section data, teacher assignments, enrollments, marklists, mark validation, published-student performance calculation, and the exact 10-star formula `sum(earned marks) / sum(maximum points) * 10`.
- Admin global posts; student grade/section feed isolation; structured student difficulty reports; optional G&C messaging; People inquiries routed to the admin inbox.
- Admin teacher 5-star rating route; teacher assignment and subject setup; XLSX marklist export and DOCX student report-card export.
- Flutter role selection, authentication surfaces, role-aware navigation, online feed integration, offline announcement cache, and logo/brand treatment.

Production work still required

This is a source-code MVP foundation, not a certified production deployment. Before handling real student data, complete verified recovery/email operations, refresh-token revocation and rate limiting, attachment storage and malware scanning, comprehensive API tests, Alembic migration automation, least-privilege grants and a full RLS policy review, immutable audit logging, full threaded messaging APIs, note uploads, seasonal roll-over and archives, deeper portal screens, device-level security review, and deployment/backup/monitoring. Confirm the school’ s consent, retention, safeguarding, and student-data requirements before launch.

Flutter/Dart source formatting and static analysis have been run. The sandbox does not have the Android SDK, so an APK could not be built here; install Flutter plus Android SDK/build tools and follow the commands above to build it. The FastAPI backend is separately syntax/import/endpoint checked before handoff.